Report of MBDA_GPU project

Orhan Efe Bayrak

July 25, 2025

Contents

1 Change of codes

I changed some codes-that cloned from `https://github.com/CoDaSLab/MBDA_build/tree/main` to set up the JupyterHub and local server,optimized to my computer.Here are the differencies:

- 1.1)Dockerfiles

```
FROM nvidia/cuda:12.1.1-cudnn8-runtime-ubuntu22.04

# System Requirements
RUN apt-get update && apt-get install -y \
    python3 python3-pip python3-dev python3-distutils python-is-python3 \
    git curl sudo npm nodejs octave

# Update pip
RUN python3 -m pip install --upgrade pip

# Old style node-supported 'configurable-http-proxy' for proxy
RUN npm install -g configurable-http-proxy@4.5.0

# PyTorch (Supports CUDA 12.1)
RUN python3 -m pip install \
    torch==2.2.2+cu121 torchvision==0.17.2+cu121 torchaudio==2.2.2 \
    --index-url https://download.pytorch.org/whl/cu121

# Jupyter enviroment and other dependencies.
RUN python3 -m pip install --no-cache-dir \
    jupyterhub notebook jupyterlab \
    git+https://github.com/jupyterhub/nativeauthenticator \
    dockerspawner octave-kernel jupyterhub-idle-culler \
    IPy PyYAML

# Clone project tools.
RUN git clone https://github.com/josecamachop/FCParser && \
    git clone https://github.com/josecamachop/MEDA-Toolbox --branch v1.3

# JupyterHub config
WORKDIR /etc/jupyterhub
COPY home /home
RUN ln -s /home/jupyterhub_config.py jupyterhub_config.py

# Start
ENTRYPOINT ["jupyterhub", "-f", "/etc/jupyterhub/jupyterhub_config.py"]
```

Figure 1: Changed Dockerfile

```
# syntax=docker/dockerfile:1

# Start from jupiterhub/jupiterhub
FROM jupyterhub/jupyterhub:5.0.0

# Updated/install packages
RUN apt-get update -y && \
    apt-get upgrade -y && \
    apt-get install npm nodejs python3 python3-pip git nano octave nfdump -y

RUN python3 -m pip install jupyterhub notebook jupyterlab octave-kernel jupyterhub-nativeauthenticator dockerspawner jupyterhub-idle-culler

# Install the FCParser and dependencies
WORKDIR /
RUN git clone https://github.com/josecamachop/FCParser
RUN python3 -m pip install IPy PyYAML

# Install the MEDA Toolbox and dependencies
RUN git clone https://github.com/josecamachop/MEDA-Toolbox --branch v1.3
RUN apt-get install octave-statistics -y

# Configure jupyterhub
RUN mkdir /etc/jupyterhub
WORKDIR /etc/jupyterhub

# Change this with your certificate files (or see modification of the last line)
COPY ssl/key.pem .
COPY ssl/cert.pem .

COPY home /home
RUN ln -s /home/jupyterhub_config.py jupyterhub_config.py

# Ready to go, change to last line if no certificate is used
SHELL ["/bin/sh", "-c"]
ENTRYPOINT ["jupyterhub", "-f", "/etc/jupyterhub/jupyterhub_config.py", "--ssl-key", "/etc/jupyterhub/key.pem", "--ssl-cert", "/etc/jupyterhub/cert.pem"]
#ENTRYPOINT ["jupyterhub", "-f", "/etc/jupyterhub/jupyterhub_config.py"]
```

Figure 2: Original Dockerfile

- 1.2)jupyterhub_config Files

```
c = get_config()

c.JupyterHub.authenticator_class = 'native'

import os, nativeauthenticator
c.JupyterHub.template_paths = [f"{os.path.dirname(nativeauthenticator.__file__)}/templates/"]

c.Authenticator.admin_users = {'orhan', 'codaslab'}
c.Authenticator.whitelist = {'orhan', 'codaslab'}
c.Authenticator.allow_all = True
c.Authenticator.open_signup = True

c.NativeAuthenticator.confirm_new_users = False
c.NativeAuthenticator.create_system_users = True

c.JupyterHub.concurrent_spawn_limit = 4

import pwd, subprocess

def pre_spawn_hook(spawner):
    username = spawner.user.name
    user_home = f"/home/{username}"
    example_predef_src = "/home/example_predef"
    example_predef_dst = f"{user_home}/example_predef"

    try:
        pwd.getpwnam(username)
    except KeyError:
        try:

            subprocess.run(['groupadd', '-f', 'mbda'], check=True)


            subprocess.check_call(['useradd', '-m', '-s', '/bin/bash', '-g', 'mbda', username])


            subprocess.run(['chmod', '700', user_home], check=True)


            if os.path.exists(example_predef_src):
                subprocess.run(['cp', '-r', example_predef_src, example_predef_dst], check=True)
                subprocess.run(['chown', '-R', f"{username}:mbda", example_predef_dst], check=True)


            for tool in ['MEDA-Toolbox', 'FCParser', 'UGR-16']:
                source = f"/{tool}"
                target = f"{user_home}/{tool}"
                if os.path.exists(source) and not os.path.exists(target):
                    subprocess.run(['ln', '-s', source, target], check=True)

        except subprocess.CalledProcessError as e:
            spawner.log.error(f"User creation failed for {username}: {e}")
            raise

c.Spawner.pre_spawn_hook = pre_spawn_hook
```

Figure 3: Changed jupyterhub_config

```
c = get_config()

c.JupyterHub.authenticator_class = 'native'

import os, nativeauthenticator
c.JupyterHub.template_paths = [f"{os.path.dirname(nativeauthenticator.__file__)}/templates/"]

c.Authenticator.admin_users = {'orhan', 'codaslab'}
c.Authenticator.whitelist = {'orhan', 'codaslab'}
c.Authenticator.allow_all = True
c.Authenticator.open_signup = True

c.NativeAuthenticator.confirm_new_users = False
c.NativeAuthenticator.create_system_users = True

c.JupyterHub.concurrent_spawn_limit = 4

import pwd, subprocess

def pre_spawn_hook(spawner):
    username = spawner.user.name
    user_home = f"/home/{username}"
    example_predef_src = "/home/example_predef"
    example_predef_dst = f"{user_home}/example_predef"

    try:
        pwd.getpwnam(username)
    except KeyError:
        try:
            subprocess.run(['groupadd', '-f', 'mbda'], check=True)
            subprocess.check_call(['useradd', '-m', '-s', '/bin/bash', '-g', 'mbda', username])
            subprocess.run(['chmod', '700', user_home], check=True)

            if os.path.exists(example_predef_src):
                subprocess.run(['cp', '-r', example_predef_src, example_predef_dst], check=True)
                subprocess.run(['chown', '-R', f"{username}:mbda", example_predef_dst], check=True)

            for tool in ['MEDA-Toolbox', 'FCParser', 'UGR-16']:
                source = f"/{tool}"
                target = f"{user_home}/{tool}"
                if os.path.exists(source) and not os.path.exists(target):
                    subprocess.run(['ln', '-s', source, target], check=True)

        except subprocess.CalledProcessError as e:
            spawner.log.error(f"User creation failed for {username}: {e}")
            raise

c.Spawner.pre_spawn_hook = pre_spawn_hook
```

Figure 4: Original jupyterhub_config

- 1.3)build.sh Files

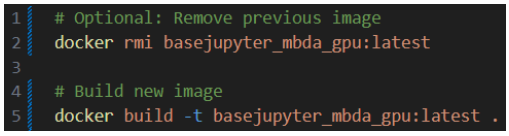

Figure 5: Changed build.sh

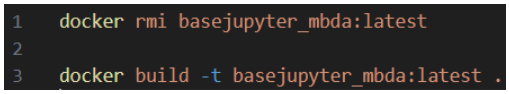

Figure 6: Original build.sh

2 File Comparisions

- 2.1)Dockerfile Comparision

Feature	**Original Dockerfile**	**Changed Dockerfile**
Base image	jupyterhub/jupyterhub:5.0.0	nvidia/cuda:12.1.1-cudnn8-runtime-ubuntu22.04
PyTorch installation	Not specified (likely CPU version)	Explicitly installs CUDA-enabled PyTorch (torch==2.2.2+cu121)
Proxy setup	Not manually handled	Installs configurable-http-proxy via npm (required by JupyterHub)
NativeAuthenticator	Installed via PyPI (jupyterhub-nativeauthenticator)	Installed directly from GitHub (possibly newer version)
SSL configuration	Yes (copies key.pem and cert.pem for HTTPS)	No SSL, runs on plain HTTP
Purpose	Basic JupyterHub setup (CPU-focused)	Full JupyterHub setup with GPU and machine learning libraries

- 2.2)jupyterhub config Comparision

Feature / Line	Original jupyterhub config	Changed jupyterhub config
template_paths setting	Not defined	Custom template path using nativeauthenticator.__file__
Admin users / whitelist	Only admin_users	Both admin_users and whitelist
open_signup	Not defined	Set to True
allow_all	Not defined	Set to True
confirm_new_users	Not defined	Set to False
create_system_users	Not defined	Set to True
Concurrent spawn limit	Not defined	Set to 4
pre_spawn_hook logic	Simple: useradd, chmod, cp, chown	Extended: groupadd, symbolic links, group checks
Example directory copy	Basic copy from /home/example_predef	Copies and sets ownership of example directory

3 Local server layout

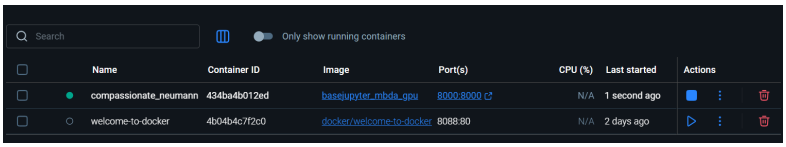

Figure 7: Docker desktop

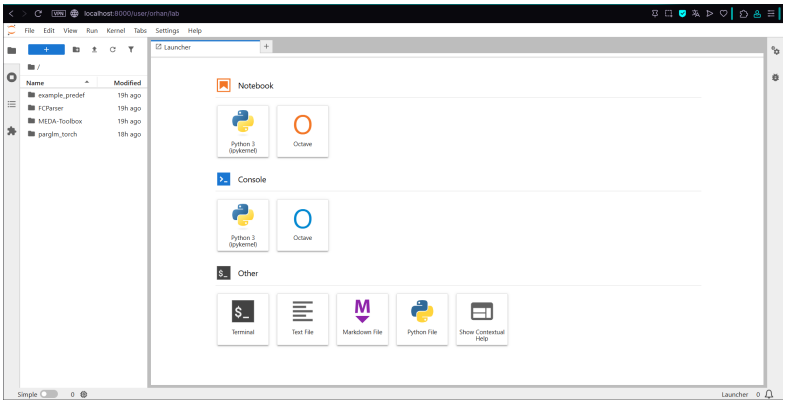

Figure 8: Interface layout

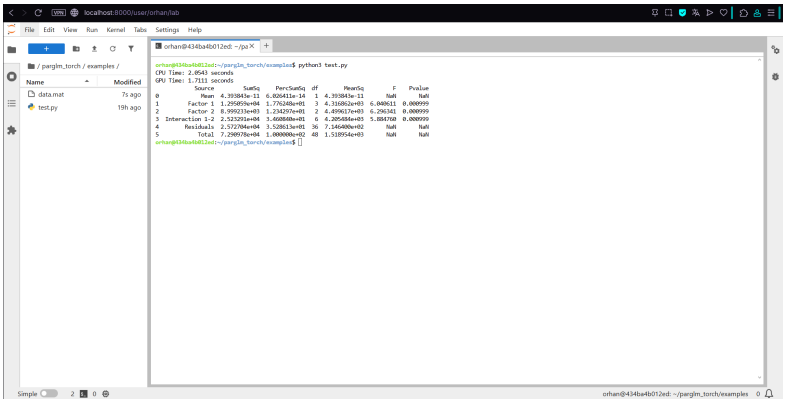

Figure 9: Run of parglm_torch example

4 Project Setup Guide (MBDA_GPU)

This section explains how to install and run the `MBDA_GPU` project on your local machine using Docker and JupyterHub.

Prerequisites

Before you begin, ensure the following are installed on your system:

- **Docker Desktop** (must remain active during build and execution)
- **NVIDIA Container Toolkit** (for GPU access)
- **Git**

Step-by-Step Setup

Setup Instructions (Quick Run)

Follow the steps below to build and launch the MBDA_GPU project using Docker:

Listing 1: Terminal Commands for Setup

```
# Step 1: Clone the repository
git clone https://github.com/Orhanbayrak/MBDA_GPU.git
cd MBDA_GPU

# Step 2: Build the Docker image
docker build -t basejupyter_mbda_gpu:latest .
```

```
# Step 3: Run the container
docker run -p 8000:8000 basejupyter_mbda_gpu:latest
```

Once the container is running, open your browser and navigate to:

http://localhost:8000

Access and Use

- Register and log in through the JupyterHub interface.
- If you will get following error: A module that was compiled using NumPy 1.x cannot be run in NumPy 2.1.2 as it may crash. To support both 1.x and 2.x versions of NumPy, modules must be compiled with NumPy 2.0. Some module may need to rebuild instead e.g. with 'pybind11¿=2.12'.

  It is because some PyTorch versions do not support Numpy 2.You need to downgrade the numpy version with following terminal command:pip install numpy==1.26.4 –force-reinstall –no-cache-dir

Notes

- The Docker image includes all necessary packages (e.g., CUDA-enabled PyTorch, GNU Octave, JupyterHub).
- SSL is not enabled by default; the container runs over plain HTTP.
- Ensure your GPU is supported and that the appropriate NVIDIA drivers are installed on the host machine.

5 NVIDIA-Docker Package

`nvidia-docker` is a tool that allows Docker containers to access the GPU (graphics card) of the host machine. Normally, Docker containers use only the CPU, but for machine learning and deep learning projects, GPU access is essential.

Why It Is Needed

- Provides GPU access inside containers.
- Automatically detects installed CUDA and NVIDIA drivers.
- Supports different CUDA versions for different projects.
- Enables PyTorch and similar libraries to use the GPU inside containers.

Installation Steps

1. Make sure the NVIDIA driver is installed on your system.
2. Install Docker.
3. Run the following commands:

```
sudo apt-get install -y nvidia-container-toolkit
sudo systemctl restart docker
```

How to Use It

After installation, you can run a GPU-enabled container like this:

```
docker run --gpus all nvidia/cuda:12.1.1-base nvidia-smi
```

Relevance to This Project

In this project, the JupyterHub server runs inside a Docker container. To enable GPU support for model training and fast computations, `nvidia-docker` must be properly installed and configured.